

CPU_VISION_RESEARCH

CPU/Metal-viable vision models for §11.4.153 — deep web research

Revision: 2 **Last modified:** 2026-06-16T09:50:00Z **Authority:** §11.4.150 (deep multi-angle web research per issue), §11.4.99 (latest-source cross-reference), §11.4.123 (rock-solid-proof-or-research), §11.4.8 (deep-web-research-before-implementation). **Companion of:** FINDINGS.md (the vision-path resolution doc).

Why this exists

FINDINGS.md (Rev 2/3) concluded "no CPU-viable vision model — moondream too slow + hallucinates; analysis stays the agent's own native multimodal read." A §11.4.150 deep-research pass (2026-06-16, subagent-driven) re-examined that conclusion from four angles and found it was correct **only for the exact path tested** (moondream: latest via Ollama) — NOT a real ceiling.

Root cause of the moondream failure (research-supported, one caveat)

The moondream: latest Ollama failure (160 s → bbox-only [0.12, 0.13, 0.87, 0.86]; 300 s timeout) is explained by two compounding documented problems, not a model limit:

1. **Ollama ships a stale (2024) moondream build.** Its tags were last updated ~2 years ago; an open request ([ollama/ollama#8391](https://github.com/ollama/ollama/issues/8391)) asks Ollama to ship the 2025-01-09 revision that "support[s] bbox and gaze detection."
2. **Ollama exposes moondream as generic text+image chat — no skill endpoints.** Modern moondream2 has distinct methods `caption()`, `query()`, `detect()` → `boxes`, `point()`, `segment()`. A free-form chat prompt into the old GGUF let the **detection/grounding head** fire instead of `caption` — exactly the raw normalized bbox observed; the caption path then hit the unoptimized MPS route and timed out.

Caveat (§11.4.6 honesty): no Ollama maintainer post *verbatim* says "we misroute to detect." This is inferred from (i) the confirmed stale build date, (ii) the lack of skill endpoints, (iii) the bbox-shaped output. Strongly supported, not definitive.

Comparison (Apple-Silicon / Metal, no CUDA, no cloud)

Model	Runtime	~Latency/ frame (Apple-Silicon)	Accuracy for grounded UI verdict	Install
Qwen2.5-VL-3B-Instruct-4bit	mlx-vlm (Metal)	single-digit ~15 s (proxy from tok/s + Qwen3-VL-4B ~2-5 s)	Best acc/size — OCRBench 77.1, DocVQA 93.9; built for docs/screenshots/ agentic UI	<code>pip install -U mlx-vlm; model mlx-community/Qwen2.5-VL-3B-Instruct-4bit</code>
Qwen3-VL-4B-Instruct-4bit	mlx-vlm (Metal)	~2-5 s/ image (M4 Max 1.2 s cold @448²)	newer gen, strong OCR/UI	<code>mlx-community/Qwen3-VL-4B-Instruct-4bit</code>
moondream2 (2B)	moondream pip + Photon (Metal)	~1.2-1.3 s/image (M2 0.79 req/s, M4 0.84, batch-4)	ScreenSpot F1 80.4, DocVQA 79.3, ChartQA 77.5; good for 2B	<code>pip install moondream; md.vl(local=True, model="moondream2")</code>
Qwen2-VL-2B / 2.5-VL-3B GGUF	llama.cpp llama-mtmd-cli (Metal)	no clean named-chip number (flagged)	OCRBench 65.5 / 77.1; first-party GGUF	<code>llama-server -hf ggml-org/Qwen2.5-VL-3B-Instruct-GGUF:Q4_K_M</code>
SmolVLM-500M	llama.cpp / MLX	sub-second-class	DocVQA 70.5, OCRBench 61; open license, simplest	<code>llama-server -hf ggml-org/SmolVLM-500M-Instruct-GGUF:Q8_0</code>
MiniCPM-V 2.6 (8B)	llama.cpp (Metal)	latency-risky (~5.4 s/ slice image encode on M2)	best raw OCR	<code>bartowski/MiniCPM-V-2_6-GGUF:Q4_K_M</code>
FastVLM-0.5B/ 1.5B (Apple)	MLX	TTFT 152-166 ms (M1)	0.5B DocVQA only 31.7 (weak on	<code>apple/ml-fastvlm</code>

Model	Runtime	~Latency/ frame (Apple- Silicon)	Accuracy for grounded UI verdict	Install
			dense numbers)	
Florence-2	transformers CPU	"several seconds" CPU; Metal broken	not for open-ended description (task-token seq2seq)	microsoft/ Florence-2-base

VERDICT

A genuinely viable CPU/Metal option exists — the prior "none viable" verdict was based on a broken test path, not a real ceiling.

- **Best UI/OCR fidelity (recommended):** `mlx-vlm + mlx-community/Qwen2.5-VL-3B-Instruct-4bit`.
- **Fastest + directly fixes the moondream story (~1.2 s/frame):** `pip install moondream`
 - Photon with `md.vl(local=True, model="moondream2")`, calling `.query().caption()` explicitly (never free-form chat) — drop Ollama.
- **Fully open license, dead-simple:** `SmolVLM-500M` or `Qwen2-VL-2B` via `llama.cpp`.

ON-HOST EMPIRICAL VALIDATION — DONE (2026-06-16, §11.4.123 rock-solid)

The research above was empirically validated on THIS host (Apple M3 Pro, arm64, macOS 15.5, **no CUDA**) — not trusted, MEASURED:

- **Setup:** `python3.13 -m venv /tmp/vlm_venv; pip install -U mlx-vlm (0.6.3, mlx 0.31.2, Metal); model mlx-community/Qwen2.5-VL-3B-Instruct-4bit (~2.9 GB, one-time download).`
- **Input:** the real dashboard capture `ytdlp---dashboard---20260616T091827Z.png`, downscaled to 1024px (`sips -Z 1024`).
- **Measured latency (model cached, /usr/bin/time -p): ~20.3 s/frame** (prompt ~295 tok/s, gen ~49 tok/s, peak 4.23 GB RAM) — **under the 30 s target**. First run was 855 s but that was the one-time model download, not compute.
- **Grounded + correct:** named the real nav tabs (Dashboard/Queue/History/Cookies), the Online status, the form fields (URL/QUALITY/FOLDER) + the "Add to Queue" button, and **all 16 platform tiles** correctly.
- **Honest flaw (§11.4.6):** it also invented plausible-but-not-visible dropdown option lists (QUALITY/FOLDER values) — i.e. it genuinely SEES the image but will occasionally fabricate plausible detail. Full-res (un-downscaled) was slightly worse; downscaling to ~1024px both speeds it up AND improves grounding.

Reproducible command (model cached on this host):

```
source /tmp/vlm_venv/bin/activate
sips -Z 1024 "<ytdlp---dashboard---*.png>" --out /tmp/vlm_dash.png
python -m mlx_vlm.generate --model mlx-community/Qwen2.5-VL-3B-Instruct-4bit \
    --image /tmp/vlm_dash.png --max-tokens 400 --temperature 0.0 \
    --prompt "Describe this dashboard UI: nav tabs, form fields, button labels, platform names, status text."
```

Conclusion (rock-solid): a local CPU/Metal vision model DOES work here (~20 s, grounded) — the prior "no CPU-viable option" verdict is overturned. **BUT** because it can fabricate plausible detail, it is a *good-not-flawless* second-opinion / scale option, NOT a replacement for the **native-multimodal read** (the agent's own zero-hallucination-on-what-it-claims read), which stays the PRIMARY §11.4.153 verdict path. The mlx_vlm path is now available for an independent ensemble or high-volume pre-screen when the operator wants analysis off the conductor. The latency-risk caveat (image encoding on Metal; downscale large captures) stands.

Recommended install commands

```
# Option A (recommended): mlx-vlm + Qwen2.5-VL-3B
pip install -U mlx-vlm
python -m mlx_vlm.generate \
    --model mlx-community/Qwen2.5-VL-3B-Instruct-4bit \
    --image /path/to/screenshot.png \
    --prompt "Describe this dashboard UI: panels, headings, metrics+values, button labels, layout." \
    --max-tokens 512 --temperature 0.0

# Option B (fastest, fixes moondream): moondream pip + Photon (NOT Ollama)
pip install moondream
python - <<'PY'
import moondream as md
from PIL import Image
m = md.vl(local=True, model="moondream2")
print(m.query(Image.open("dashboard.png"), "Describe this dashboard: chart titles, key metrics, button labels.")
      ["answer"])
PY
```

Sources verified 2026-06-16

machinelearning.apple.com/research/fast-vision-language-models · arxiv.org/html/2412.13303v1 · github.com/apple/ml-fastvlm · arxiv.org/html/2504.05299v1 · huggingface.co/blog/smolvlm2 · huggingface.co/vikhyatk/moondream2 · docs.moondream.ai/running-locally · pypi.org/project/moondream · moondream.ai/blog/photon-1-2-0-update · ollama.com/library/moondream/tags · github.com/ollama/ollama/issues/8391 · github.com/ggml-org/llama.cpp/blob/master/tools/mtmd/README.md · github.com/ggml-org/llama.cpp/issues/14527 ·

huggingface.co/ggml-org/Qwen2.5-VL-3B-Instruct-GGUF · huggingface.co/bartowski/MiniCPM-V-2_6-GGUF · qwenlm.github.io/blog/qwen2.5-vl · github.com/Blaizzy/mlx-vlm · huggingface.co/mlx-community/Qwen2.5-VL-3B-Instruct-4bit · huggingface.co/microsoft/Florence-2-large-ft/discussions/4 · huggingface.co/qnguyen3/nanoLLaVA